

Inteligencia Artificial

Informe Final: Problema Multi Depot Vehicle Routing Problem with Time Windows

Pablo Retamales

23 de junio de 2026

Evaluación

Código Fuente (10 %):	_____
Resumen (2 %):	_____
Introducción (3 %):	_____
Representación (10 %):	_____
Descripción del algoritmo (20 %):	_____
Experimentos (10 %):	_____
Resultados (30 %):	_____
Conclusiones (10 %):	_____
Bibliografía (5 %):	_____
Nota Final (100):	_____

Resumen

El Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW) es una variante del Vehicle Routing Problem (VRP) que considera múltiples depósitos y restricciones de tiempo para la atención de clientes. El objetivo es determinar rutas de mínimo costo para una flota de vehículos que atiende clientes dentro de intervalos de tiempo definidos y retorna a su depósito de origen. Debido a su complejidad computacional, el MDVRP-TW ha sido abordado principalmente mediante metaheurísticas como búsqueda tabú y algoritmos genéticos, aunque también existen métodos exactos para instancias pequeñas. Este informe presenta una descripción general del problema, sus principales variantes, métodos de resolución y un modelo matemático de programación entera mixta.

1. Introducción

El presente informe tiene como propósito describir y analizar el Multi-Depot Vehicle Routing Problem with Time Windows (MDVRP-TW), un problema de optimización combinatoria de gran relevancia en el ámbito de la logística y la distribución. Este problema surge de la necesidad de coordinar flotas de vehículos que operan desde múltiples centros de distribución para atender a un conjunto de clientes con intervalos de tiempo definidos, buscando minimizar el costo total de las rutas.

En este informe se presenta la definición del problema, sus variables, parámetros, restricciones y objetivo, además de una revisión de los métodos utilizados para resolverlo. A través de este análisis, el informe busca consolidar el estado del arte del problema para identificar cuáles son las estrategias de resolución más eficientes y detectar las brechas metodológicas que

guiarán el desarrollo de trabajos futuros. Posteriormente, se expone su modelo matemático de programación entera mixta y se discuten tendencias actuales de investigación.

El interés por estudiar el MDVRP-TW se debe a su aplicación en sistemas reales de distribución y transporte, utilizados por empresas de logística como Amazon. El problema es de naturaleza NP-hard, ya que generaliza problemas de optimización combinatoria conocidos por ser NP-hard, como el problema del vendedor viajero (TSP) y sus variantes de ruteo de vehículos. Además, integra simultáneamente decisiones de asignación de clientes a depósitos, planificación de rutas y cumplimiento de ventanas de tiempo, lo que incrementa significativamente la complejidad del problema. La interacción entre estos subproblemas provoca un crecimiento combinatorio del espacio de búsqueda de soluciones, dificultando la obtención del óptimo global en tiempos computacionales razonables para instancias de gran tamaño. Por ello, el desarrollo de métodos capaces de encontrar soluciones de alta calidad en tiempos acotados sigue siendo un desafío relevante dentro del área de optimización combinatoria.

Para abordar este desafío, el acercamiento propuesto consiste en el diseño e implementación de la técnica de Simulated Annealing, enfocada en la exploración eficiente del espacio de búsqueda mediante múltiples operadores de vecindad. El comportamiento del algoritmo se evalúa bajo un entorno experimental estructurado sobre instancias desde 25 hasta 1000 clientes, permitiendo ajustar sus parámetros críticos. En cuanto a la estructura del documento, la Sección 2 detalla la definición general del problema; la Sección 3 revisa el estado del arte; la Sección 4 expone el modelo matemático formal; la Sección 5 describe la representación de soluciones; la Sección 6 profundiza en el algoritmo implementado; la Sección 7 detalla la metodología de la experimentación, la Sección 8 presenta los resultados de la experimentación y, finalmente, la Sección 9 presenta las conclusiones.

2. Definición del Problema

El Vehicle Routing Problem (VRP) es un problema de optimización combinatoria que consiste en determinar el conjunto de rutas de menor costo para una flota de vehículos que parte desde un depósito central para atender a un conjunto de clientes, retornando al mismo depósito [13]. El MDVRP-TW extiende esta formulación al considerar múltiples depósitos y ventanas de tiempo para la atención de los clientes.

Dado un grafo completo $G = (V, A)$, donde $V = D \cup C$ es el conjunto de todos los nodos, siendo D el conjunto de depósitos y C el conjunto de clientes, y A el conjunto de arcos (i, j) entre todos los pares de nodos, el objetivo es determinar un conjunto de rutas para una flota de vehículos tal que se minimice el costo total de distribución, que comprende la distancia recorrida, el tiempo empleado y las penalizaciones por incumplimiento de las ventanas de tiempo [6].

Cada ruta debe comenzar y terminar en el mismo depósito, cada cliente debe ser visitado exactamente una vez, la demanda acumulada (demanda total) de los clientes en cada ruta no puede superar la capacidad del vehículo, y cada cliente debe ser atendido dentro de su ventana de tiempo $[e_i, l_i]$.

El objetivo del problema, de forma explícita, es determinar un conjunto de rutas para la flota de vehículos que permita atender a todos los clientes respetando sus restricciones de capacidad y de ventanas de tiempo, de manera que el costo total de operación sea lo más bajo posible. Este costo incluye principalmente la distancia recorrida, el tiempo de desplazamiento y posibles penalizaciones asociadas al incumplimiento de los intervalos de atención de los clientes.

2.1. Variables de Decisión

Las principales decisiones que deben determinarse en el MDVRP-TW son:

- Determinar qué vehículo atenderá a cada cliente y el orden en que se realizarán las visitas.

- Establecer los tiempos de llegada e inicio de servicio en cada cliente.
- Calcular los posibles retrasos respecto a las ventanas de tiempo establecidas, los cuales pueden generar penalizaciones.

2.2. Restricciones

Las restricciones fundamentales del MDVRP-TW son las siguientes:

- Cada cliente debe ser atendido exactamente una vez.
- Todos los vehículos deben iniciar su recorrido desde el depósito que les ha sido asignado.
- La demanda total de los clientes atendidos en una ruta no puede superar la capacidad del vehículo.
- Los vehículos deben regresar a su depósito de origen dentro de los intervalos de tiempo establecidos; en caso contrario, se aplican las penalizaciones correspondientes.
- Si un vehículo llega antes de la apertura de la ventana de tiempo de un cliente, debe esperar hasta el inicio del servicio. Si llega después del límite establecido, se aplica una penalización proporcional al retraso.

2.3. Problemas Relacionados y Variantes

El MDVRP-TW se enmarca dentro de la familia de problemas VRP. Las variantes más relevantes incluyen:

- **Capacited Vehicle Routing Problem (CVRP)**: solo considera restricción de capacidad, sin ventanas de tiempo ni múltiples depósitos.
- **Vehicle Routing Problem with Time Windows (VRPTW)**: incorpora ventanas de tiempo, pero con un único depósito.
- **Multi-Depot Vehicle Routing Problem (MDVRP)**: considera múltiples depósitos sin restricciones de tiempo [15].
- **Multi-Depot Open Vehicle Routing Problem with Time Windows (MDOVRPTW)**: variante abierta, donde los vehículos no necesitan retornar al depósito de origen [10].
- **Multi-Depot Dynamic Vehicle Routing Problem with Time Windows (MD-DVRPTW)**: incorpora demanda dinámica de clientes en tiempo real [17].

3. Estado del Arte

El VRP fue introducido por Dantzig y Ramser en 1959 en el contexto de la distribución de gasolina desde un depósito central a múltiples estaciones [4]. La incorporación de ventanas de tiempo al VRP (VRPTW) fue formalizada y sistematizada por Solomon en 1987, quien propuso tanto heurísticas de construcción basadas en inserción como instancias de prueba que se han convertido en un estándar para evaluar algoritmos dentro del VRP [12]. La extensión al caso multi-depósito fue formalizada por Cordeau et al. en 1997 con el MDVRP, siendo extendida en 2001 al MDVRP-TW mediante una Búsqueda Tabú Unificada (Unified Tabu Search, UTS) [2].

3.1. Representación de Soluciones

En la resolución del MDVRP-TW, la elección de la estructura de datos para codificar las rutas es crítica para el desempeño del algoritmo. El estado del arte clasifica las representaciones principalmente en dos enfoques:

- **Representación Directa (Multi-ruta):** Codifica las soluciones como una lista explícita de rutas asociadas a cada depósito, delimitadas por el origen y fin de cada viaje. Permite una evaluación inmediata de las restricciones de capacidad y ventanas de tiempo, facilitando la aplicación directa de operadores de vecindad locales como *Swap* o *Relocate* [2].
- **Representación Indirecta (Basada en Permutaciones):** Consiste en una secuencia continua de clientes sin delinear los vehículos ni los depósitos. Este enfoque delega la construcción física de las rutas a un algoritmo decodificador (como el procedimiento Split, introducido por Prins y adaptado para problemas con ventanas de tiempo por Vidal et al. [14]), el cual segmenta la permutación de forma óptima introduciendo los depósitos y vehículos requeridos. Es la representación predilecta en algoritmos genéticos para evitar la generación de descendencia inválida tras los cruces.

3.2. Métodos Exactos

Los métodos exactos para el MDVRP-TW, tales como Branch-and-Bound, Branch-and-Cut y generación de columnas, permiten obtener soluciones óptimas garantizadas. Sin embargo, su aplicabilidad práctica se limita a instancias de pequeño tamaño debido al crecimiento exponencial del espacio de búsqueda. La naturaleza NP-hard del problema hace que estos métodos sean inviables para instancias grandes con decenas de depósitos y cientos de clientes [13].

3.3. Búsqueda Tabú (TS)

La Búsqueda Tabú (Tabu Search, TS) ha sido históricamente uno de los métodos más efectivos para resolver el MDVRP-TW. Cordeau et al. [2] propusieron una heurística tabú unificada que maneja simultáneamente el VRPTW, el MDVRP-TW y el PVRPTW. El método utiliza movimientos de reubicación de clientes entre rutas y una función objetivo penalizada que permite aceptar soluciones infactibles de manera temporal. Este enfoque, basado en el manejo de restricciones blandas mediante penalización, facilita la exploración de regiones del espacio de búsqueda que de otro modo serían difíciles de alcanzar. Posteriormente, Cordeau y Maischberger [3] mejoraron este enfoque mediante búsqueda tabú iterada en paralelo, aprovechando arquitecturas de múltiples núcleos para reducir tiempos de cómputo.

Desde una perspectiva empírica, este enfoque demostró que el rendimiento de la búsqueda está estrechamente ligado al número de iteraciones fijadas. Para el MDVRP-TW, una ejecución estándar de 10^5 iteraciones permite alcanzar soluciones de alta calidad con un gap promedio inferior al 1,5 % respecto a las mejores soluciones conocidas (Best-known solutions, BKS), requiriendo tiempos de cómputo de entre 5 minutos y un poco más de 3 horas dependiendo de la dimensión de la instancia. Asimismo, el algoritmo demuestra ser altamente eficiente para configuraciones rápidas, logrando soluciones factibles con un gap promedio por debajo del 5 % en menos de 20 minutos al limitar la búsqueda a solo 10^4 iteraciones [3].

3.4. Búsqueda por Vecindad Variable (VNS)

La Búsqueda por Vecindad Variable (Variable Neighborhood Search, VNS) fue aplicada al MDVRP-TW por Polacek et al. [9], obteniendo resultados comparables y competitivos con la Búsqueda Tabú tradicional. El método explora vecindarios de tamaño creciente a partir de la solución actual, lo que permite escapar de óptimos locales mediante cambios sistemáticos de

vecindario. Sadati et al. [10] propusieron una variante denominada Variable Tabu Neighborhood Search (VTNS), que incorpora elementos de búsqueda tabú durante la fase de perturbación del VNS. Desde una perspectiva empírica, al evaluar esta metodología sobre las 20 instancias de referencia de Cordeau, el algoritmo VTNS demostró una alta precisión al alcanzar un gap promedio de 0,05 % en sus mejores soluciones respecto a las BKS vigentes en su momento, logrando igualar el récord histórico en 11 problemas y establecer 1 nueva cota superior [10].

3.5. Algoritmos Genéticos e Híbridos

Los algoritmos genéticos (AG) han sido ampliamente aplicados al MDVRP-TW. Vidal et al. [14] propusieron un algoritmo genético híbrido con manejo adaptativo de diversidad para una clase amplia de problemas VRP con ventanas de tiempo, obteniendo resultados de alta calidad gracias a la combinación de operadores de cruce ricos en información con búsqueda local intensiva.

Desde una perspectiva empírica, el análisis textual de los autores detalla que el *Hybrid Genetic Search with Advanced Diversity Control* (HGS-ADC) superó a todos los enfoques existentes del estado del arte en su momento, reportando que la inclusión de la proximidad temporal redujo el gap promedio de desviación de 0,68 % a 0,53 % frente a búsquedas puramente espaciales. Asimismo, las pruebas estadísticas de Wilcoxon ($p = 0,000$) y Levene ($p = 0,011$) confirmaron con alta significancia una mejora crítica en la estabilidad de las soluciones y una escalabilidad capaz de resolver instancias masivas de hasta 1000 clientes [14].

En trabajos recientes han explorado hibridaciones con búsqueda por gran vecindario. El Hybrid Adaptive Large Neighborhood Search (HALNS) de Voigt et al. [16] combina la capacidad exploratoria del ALNS con la recombinación de soluciones propia de los algoritmos genéticos, mostrando buen desempeño en instancias de referencia del MDVRP y problemas relacionados.

3.6. Búsqueda por Gran Vecindario Adaptativa (ALNS)

La Búsqueda por Gran Vecindario Adaptativa (Adaptive Large Neighborhood Search, ALNS), introducida por Ropke y Pisinger (2006), ha sido ampliamente utilizada en la resolución de variantes del VRP, incluyendo el MDVRP-TW. El ALNS alterna de manera adaptativa entre múltiples operadores de destrucción y reparación, asignando mayor peso a aquellos que han mostrado mejor desempeño histórico durante la búsqueda [7]. Wang et al. [17] aplicaron una variante del ALNS al problema dinámico multi-depósito con ventanas de tiempo (MD-DVRPTW), incorporando operadores de eliminación innovadores y un método de inserción basado en compatibilidad de ventanas de tiempo. Desde una perspectiva empírica, el ALNS ha demostrado una alta efectividad en problemas con restricciones temporales, logrando identificar históricamente 48 nuevas soluciones de referencia BKS. Esto valida la capacidad de su estructura adaptativa para manejar la complejidad en escenarios que, como la variante de Wang et al. [17], escalan con éxito hasta los 288 clientes mediante un mecanismo de inserción en tiempo constante $O(1)$.

3.7. Aceleración mediante GPU

Una tendencia reciente en la resolución del MDVRP-TW es el uso de unidades de procesamiento gráfico (GPU) para acelerar la evaluación de vecindarios dentro de metaheurísticas. Este enfoque permite explotar el paralelismo masivo de las GPU para reducir significativamente los tiempos de cómputo en instancias de gran escala.

Trabajos recientes han propuesto esquemas de aceleración tensorial basados en GPU para problemas de ruteo multi-depósito, integrando evaluaciones eficientes de movimientos y estrategias de actualización multi-movimiento tipo *leader-follower* (leader-follower strategy). Estas técnicas permiten explorar simultáneamente múltiples vecindarios, mejorando tanto la eficiencia computacional como la calidad de las soluciones obtenidas.

En este contexto, se han reportado mejoras significativas en instancias de referencia del MDVRP-TW mediante el algoritmo híbrido multifactorial por descomposición (MDFIHA), incluyendo la obtención de nuevas cotas superiores en problemas de pequeña y gran escala, evidenciando el potencial de estas técnicas para acelerar metaheurísticas modernas sin degradar el desempeño de la búsqueda [5].

Específicamente, en las instancias clásicas C01, la variante MDFIHA logra un gap promedio de -0.03 % y establece 4 nuevas cotas superiores (BKS). En las instancias de gran escala V13, a pesar de emplear un límite de tiempo menor (7,200 s frente a los 12,960 s de enfoques en CPU), MDFIHA alcanza un gap de -0.22 % mejorando 19 BKS, mientras que su variante acelerada por GPU con algoritmos genéticos (MDFIHA-ETGA) potencia este rendimiento con un gap de -0.60 % y 26 nuevas soluciones récord. Se comprobó además que estas mejoras en la calidad de las soluciones y tiempos de cómputo son estadísticamente significativas frente a los algoritmos de referencia ($p < 0,05$) [5].

3.8. Variantes Modernas y Tendencias

La investigación reciente ha extendido el MDVRP-TW hacia problemas con mayor realismo:

- **Ventanas de tiempo blandas:** permiten relajar restricciones de tiempo mediante penalizaciones [6].
- **MDVRP-TW con emisión de carbono:** incorpora costos de emisión y cuotas de comercio de carbono en la función objetivo [11].
- **MDVRP-TW con demanda dinámica:** los clientes pueden realizar pedidos en tiempo real durante la ejecución de las rutas [17].
- **MDVRP-TW con depósito flexible:** los vehículos pueden retornar a un depósito distinto del de partida, reduciendo la distancia total recorrida [8].

La tendencia actual se orienta hacia métodos híbridos que combinan búsqueda local intensiva con estrategias para mantener la diversidad de soluciones. Además, se han propuesto enfoques basados en aprendizaje automático para predecir configuraciones de parámetros adecuadas en metaheurísticas aplicadas a problemas de ruteo [1].

4. Modelo Matemático

A continuación se presenta un modelo de programación entera mixta (MIP) para el MDVRP-TW, basado en la formulación de Cordeau et al. [2] y extendido según Li y He [6].

4.1. Conjuntos, Parámetros y Variables

Sea $V = D \cup C$ el conjunto de todos los nodos, con $D = \{n + 1, \dots, n + m\}$ los depósitos y $C = \{1, \dots, n\}$ los clientes. Se define K como el conjunto de todos los vehículos, donde cada vehículo $k \in K$ está asociado a un depósito de origen $o(k) \in D$.

Variables de decisión:

- $x_{ijk} \in \{0, 1\}$: toma el valor 1 si el vehículo k recorre el arco (i, j) , 0 en caso contrario.
- $t_{ik} \geq 0$: tiempo de inicio de servicio del vehículo k en el nodo i .
- $w_{ik} \geq 0$: penalización por violación de ventana de tiempo del vehículo k en el nodo i .

Parámetros del problema:

- $D = \{d_1, d_2, \dots, d_m\}$: conjunto de depósitos, con m depósitos en total.
- $C = \{c_1, c_2, \dots, c_n\}$: conjunto de clientes, con n clientes en total.
- k : número máximo de vehículos disponibles por depósito.
- Q : capacidad máxima de carga de cada vehículo.
- q_i : demanda del cliente $i \in C$.
- $[e_i, l_i]$: ventana de tiempo válida en la que puede ser atendido un cliente i .
- s_i : tiempo de servicio del cliente i .
- d_{ij} : distancia euclidiana entre los nodos i y j .
- v : velocidad constante de los vehículos.
- α : coeficiente de penalización por minuto fuera de la ventana de tiempo.
- M : constante suficientemente grande utilizada en las restricciones temporales (Big-M).

4.2. Función Objetivo

Se minimiza el costo total, compuesto por distancia recorrida y penalizaciones por incumplimiento de ventanas de tiempo:

$$\min \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} d_{ij} \cdot x_{ijk} + \alpha \sum_{k \in K} \sum_{i \in C} w_{ik} \quad (1)$$

donde d_{ij} es la distancia entre los nodos i y j , y α es el coeficiente de penalización por minuto fuera de la ventana de tiempo.

El primer término minimiza la distancia total recorrida por la flota, mientras que el segundo incorpora penalizaciones asociadas al incumplimiento de las ventanas de tiempo.

4.3. Restricciones

(1) Cada cliente es visitado exactamente una vez: Esta restricción asegura que cada cliente sea atendido por un único vehículo proveniente de un único depósito, garantizando que se satisfaga la totalidad de la demanda sin duplicidad de visitas.

$$\sum_{k \in K} \sum_{j \in V} x_{ijk} = 1 \quad \forall i \in C \quad (2)$$

(2) Conservación de flujo en cada nodo: Establece la continuidad de las rutas e indica que si un vehículo k visita a un cliente i , tiene la obligación estricta de salir de dicho nodo hacia cualquier otro destino j , impidiendo rutas inconclusas.

$$\sum_{j \in V} x_{ijk} - \sum_{j \in V} x_{jik} = 0 \quad \forall i \in C, \forall k \in K \quad (3)$$

(3) Cada vehículo parte de su depósito de origen: Determina las condiciones de inicio de las rutas, indicando que cada vehículo k puede salir a lo sumo una vez desde el depósito que tiene asignado como origen, denotado por $o(k)$.

$$\sum_{j \in C} x_{o(k),j,k} \leq 1 \quad \forall k \in K \quad (4)$$

(4) Restricción de capacidad: Restringe la carga total transportada, obligando a que la suma de las demandas individuales q_i de los clientes asignados en la secuencia de recorrido de un vehículo k no exceda la capacidad máxima Q de la flota.

$$\sum_{i \in C} q_i \sum_{j \in V} x_{ijk} \leq Q \quad \forall k \in K \quad (5)$$

(5) Consistencia temporal (propagación de tiempos): Modela de forma lineal la secuencia del cronograma. Si el vehículo k transita directamente desde i hacia j ($x_{ijk} = 1$), el tiempo de llegada a j (t_{jk}) debe ser mayor o igual al arribo en i (t_{ik}), más su respectivo tiempo de servicio s_i y el tiempo de viaje determinado por la distancia d_{ij} y la velocidad v . Si $x_{ijk} = 0$, la ecuación queda desactivada mediante el valor M .

$$t_{ik} + s_i + \frac{d_{ij}}{v} \leq t_{jk} + M(1 - x_{ijk}) \quad \forall i, j \in V, \forall k \in K \quad (6)$$

donde M es un valor suficientemente grande (constante de Big-M).

(6) Penalización por violación de la ventana de tiempo: Estas ecuaciones definen la variable de holgura no negativa w_{ik} , la cual cuantifica el retraso temporal incurrido por el vehículo k en caso de llegar después del límite superior permitido (l_i) por el cliente i .

$$w_{ik} \geq t_{ik} - l_i \quad \forall i \in C, \forall k \in K \quad (7)$$

$$w_{ik} \geq 0 \quad \forall i \in C, \forall k \in K \quad (8)$$

(7) Espera por ventana de tiempo temprana: Obliga a que si un vehículo arriba a un nodo antes de la hora de apertura permitida (e_i), el inicio del servicio se retrase hasta que la ventana horaria se active formalmente. El cálculo solo tiene efecto si el nodo i es visitado.

$$t_{ik} \geq e_i \cdot \sum_{j \in V} x_{jik} \quad \forall i \in C, \forall k \in K \quad (9)$$

(8) Naturaleza de las variables: Establece el dominio matemático del modelo, restringiendo las variables de ruteo x_{ijk} a valores binarios y los tiempos continuos de llegada t_{ik} a valores no negativos.

$$x_{ijk} \in \{0, 1\} \quad \forall i, j \in V, \forall k \in K \quad (10)$$

$$t_{ik} \geq 0 \quad \forall i \in V, \forall k \in K \quad (11)$$

4.4. Espacio de Búsqueda

El espacio de búsqueda del modelo anterior está determinado principalmente por las variables binarias x_{ijk} . Con n clientes, m depósitos y $|K|$ vehículos, el número de variables binarias es del orden de $O((n + m)^2 \cdot |K|)$, lo que para instancias medianas (e.g., $n = 100$, $m = 5$, $|K| = 20$) representa del orden de 10^5 variables binarias. Dado que el problema es NP-hard, el espacio de soluciones factibles crece de manera superpolinomial con el tamaño de la instancia, justificando el uso de metaheurísticas para instancias de mediano y gran tamaño.

5. Representación

Para abordar el MDVRP-TW mediante la técnica de Simulated Annealing, se define una representación estructurada basada en un vector de rutas activas. Cada componente de este vector corresponde a un objeto que modela de forma independiente la trayectoria de un vehículo,

el cual está compuesto por un identificador del depósito (`int depot`) que especifica su origen y retorno, y una secuencia de clientes (`vector<int>`) que almacena de manera ordenada los índices de los nodos visitados. Por ejemplo, en un escenario con dos depósitos, una solución en memoria se estructura mediante un vector principal con elementos como Ruta 1: [`depot` = 1, `clientes` = [3, 5, 1]] y Ruta 2: [`depot` = 2, `clientes` = [4, 2]].

Esta representación ofrece ventajas críticas para el desempeño del algoritmo. En primer lugar, la asociación directa del depósito dentro de cada ruta permite validar la pertenencia geográfica y calcular distancias euclidianas sin mapeos externos. En segundo lugar, al agrupar los clientes secuencialmente en un vector dinámico, la verificación de la capacidad y la propagación temporal para las ventanas de tiempo $[e_i, l_i]$ se realizan mediante un recorrido lineal simple, optimizando el tiempo de cómputo por iteración. Finalmente, esta estructura agiliza la aplicación de operadores de vecindad, ya que los movimientos locales como transferencias o intercambios modifican directamente los vectores internos de las rutas seleccionadas sin alterar el resto del sistema.

6. Descripción del algoritmo

La resolución del MDVRP-TW se aborda mediante una implementación de la técnica Simulated Annealing (SA). El algoritmo comienza fijando una solución inicial como punto de partida en el espacio de búsqueda. A partir de este estado, el proceso opera bajo un esquema iterativo controlado por un ciclo fijo que ejecuta un número determinado de iteraciones máximas (*MaxIteraciones*). En cada iteración se selecciona estocásticamente un movimiento de vecindad aplicando probabilidades específicas: existe un 70 % de probabilidad de realizar un intercambio interno de clientes en una misma ruta (**Swap**), un 20 % de mover un cliente hacia otra trayectoria (**Relocate**), y un 10 % de dividir una ruta activa para transferir sus clientes restantes a un nuevo vehículo del mismo depósito (**Split**). Estas ponderaciones asimétricas se establecieron con el propósito de equilibrar la intensificación local y la diversificación global en el espacio de búsqueda. Al asignar un sesgo mayoritario al operador **Swap** (70 %), el algoritmo prioriza la optimización interna de las secuencias ya consolidadas, buscando corregir de forma rápida los cruces geográficos y los desajustes en las ventanas de tiempo de rutas individuales sin alterar la asignación general de depósitos. Por su parte, la probabilidad del 20 % otorgada a **Relocate** actúa como un mecanismo intermedio que redistribuye la carga laboral transfiriendo clientes problemáticos entre distintas rutas. Finalmente, se reserva una probabilidad restrictiva del 10 % para el operador **Split** debido a su carácter altamente disruptivo; dado que este movimiento altera drásticamente la estructura del vector de soluciones al activar nuevos vehículos e incrementar los costos fijos de transporte, una frecuencia baja asegura que sólo se aplique para romper estancamientos mayores o abrir nuevos frentes de exploración cuando la temperatura sea lo suficientemente favorable.

Inmediatamente después de alterar las rutas, se evalúa de forma estricta su factibilidad computacional frente a las restricciones de capacidad de carga de los vehículos (Q) y los intervalos de las ventanas de tiempo $[e_i, l_i]$. Si el vecino viola cualquiera de estos criterios, se descarta el movimiento de inmediato mediante una condición de rechazo duro y se continúa a la siguiente iteración, asegurando de esta manera que el algoritmo explore de forma exclusiva el espacio de soluciones válidas. Para los vecinos factibles, se determina la diferencia de costos de las distancias euclidianas (Δ). Si el vecino disminuye el costo total, se acepta automáticamente; si representa un deterioro, se acepta de manera condicional con base en el criterio de Metrópolis utilizando una probabilidad exponencial gobernada por la temperatura actual (T). La mejor solución histórica (S_{best}) se actualiza cada vez que se halla un costo menor, restableciendo a cero el contador de estancamiento. Por su parte, la temperatura disminuye geométricamente tras transcurrir un intervalo fijo de iteraciones (*IntervaloEnfriamiento*). Finalmente, si el sistema experimenta un estancamiento prolongado superior al límite tolerado (*MaxEstancamiento*),

se activa un mecanismo de recalentamiento ligero que reajusta la temperatura del sistema al 30 % de su valor inicial ($T_{inicial}$), restituyendo transitoriamente la capacidad de aceptar peores soluciones para facilitar la salida de óptimos locales. El flujo general de este comportamiento particular se detalla en el siguiente bloque de pseudocódigo:

```

Procedimiento SimulatedAnnealing(S_inicial, instancia)
    S <- S_inicial
    S_best <- S_inicial
    T <- T_inicial
    estancamiento <- 0

    Para iter <- 1 hasta MaxIteraciones Hacer
        S_vecino <- GenerarVecino(S) // Swap (70%), Relocate (20%) o Split (10%)

        Si No EsFactible(S_vecino, instancia) entonces
            Siguierte iteración
        Fin Si

        Delta <- CalcularCosto(S_vecino) - CalcularCosto(S)

        Si (Delta < 0) O (GenerarAleatorioReal(0, 1) < e^(-Delta / T)) entonces
            S <- S_vecino
            Si CalcularCosto(S) < CalcularCosto(S_best) entonces
                S_best <- S
                estancamiento <- 0
            Fin Si
        Sino
            estancamiento <- estancamiento + 1
        Fin Si

        Si (iter es múltiplo de IntervaloEnfriamiento) entonces
            T <- T * TasaEnfriamiento
        Fin Si

        Si estancamiento > MaxEstancamiento entonces
            T <- T_inicial * 0.3
            estancamiento <- 0
        Fin Si
    Fin Para

    Retornar S_best
Fin Procedimiento

```

7. Experimentos

La metodología experimental se diseñó para evaluar el comportamiento del algoritmo de Simulated Annealing ante instancias de diferente complejidad y determinar la configuración de parámetros que optimice la calidad de las soluciones. Al tratarse de una técnica incompleta y estocástica, el rendimiento está fuertemente condicionado por los operadores probabilísticos y los valores asignados al esquema de enfriamiento, requiriendo un ajuste paramétrico secuencial.

Para el estudio se seleccionaron cuatro instancias representativas de la literatura con un número escalar de clientes que define su dimensión: una instancia muy pequeña de 25 clientes,

una pequeña de 100 clientes, una mediana de 400 clientes y una grande de 1000 clientes. La selección de este espectro permite analizar la escalabilidad del algoritmo y observar las variaciones en la propagación de restricciones de tiempo ante espacios de búsqueda masivos. Todas las pruebas fueron ejecutadas en un entorno controlado mediante el uso de archivos `Makefile` para garantizar la reproducibilidad, fijando la semilla del generador de números aleatorios (`SEED=42`).

El proceso de ajuste de parámetros se estructuró en cuatro etapas sucesivas e incrementales:

1. **Evaluación del parámetro α :** Se evaluó el impacto de este factor de ponderación variando su valor en el conjunto $\{0, 0,25, 0,5, 0,75, 1, 3, 5, 10\}$ a lo largo de las cuatro instancias definidas, manteniendo constantes los demás elementos de configuración, con el fin de identificar tendencias de convergencia según el tamaño del problema.
2. **Ajuste de la temperatura inicial (T_0):** Fijando la instancia mediana de 400 clientes y estableciendo parámetros fijos `COOLING_RATE=0.99`, `MAX_ITERATIONS=1000000`, `COOLING_INTERVAL=50` y `MAX_STAGNATION=5000`, se probaron distintas magnitudes de temperatura en el rango de 100 a 10000 para determinar la temperatura inicial adecuada.
3. **Determinación del ratio de enfriamiento (`COOLING_RATE`):** Con la temperatura inicial crítica ya seleccionada y fijada, se procedió a variar exclusivamente la tasa de descenso geométrico dentro de los valores $\{0,95, 0,97, 0,98, 0,99, 0,995\}$ para calibrar la velocidad de convergencia y el balance de aceptación de soluciones de peor calidad.
4. **Calibración del umbral de estancamiento (`MAX_STAGNATION`):** Utilizando los mejores valores consolidados en las fases previas, se modificó de forma aislada el límite de iteraciones permitidas sin mejoras experimentando en valores desde 1000 hasta 30000 iteraciones, con el fin de evaluar la capacidad del algoritmo para mitigar el estancamiento y escapar de óptimos locales mediante el recalentamiento.

8. Resultados

Los resultados obtenidos en las fases de calibración paramétrica reflejan de manera clara el comportamiento estocástico del algoritmo y el impacto crucial de equilibrar adecuadamente las restricciones frente a la función objetivo.

8.1. Análisis del parámetro 'alpha' y escalabilidad

La primera fase experimental consistió en evaluar la sensibilidad del algoritmo frente a variaciones del parámetro α , el cual pondera los componentes de la función objetivo. Los datos consolidados para las cuatro instancias de prueba se presentan en la Cuadro 1.

A partir de estos datos, se observa que el comportamiento ideal de α se desplaza en función de la dimensión del problema. Para instancias pequeñas y muy pequeñas (25 y 100 clientes), un valor de $\alpha = 0,5$ proporciona el mejor equilibrio, logrando una penalización de tiempo de cero y minimizando eficientemente la distancia total recorrida. Esto ocurre porque en espacios muestrales reducidos, penalizar moderadamente los desajustes temporales guía rápidamente la búsqueda hacia regiones factibles de alta calidad.

Sin embargo, a medida que la complejidad escala hacia las instancias mediana y grande (400 y 1000 clientes), la presión de las restricciones de ventanas de tiempo se vuelve masiva. En este escenario, valores bajos de α (como 0,25 o 0,5) provocan que la función objetivo no castigue lo suficiente la infactibilidad interna explorada, resultando en un elevado número de nodos penalizados y tiempos remanentes altos. Para estas instancias más complejas, fijar $\alpha = 1,0$ demostró ser significativamente más robusto, reduciendo sustancialmente el valor objetivo y la distancia total válida recorrida al coordinar de mejor manera la transición de vecindarios más densos.

Cuadro 1: Resultados de la evaluación del parámetro α en las cuatro instancias basales.

Instancia	α	Valor Objetivo	Nodos Pe- nalizados	Penalización Tiempo	Distancia Recorrida
(25) C101	0	141,039	19	12289,9	141,039
	0,25	213,568	1	10,6011	210,918
	0,5	162,577	0	0	162,577
	0,75	174,17	0	0	174,17
	1	182,328	0	0	182,328
	3	217,064	0	0	217,064
	5	210,982	0	0	210,982
	10	162,577	0	0	162,577
(100) C101	0	1275,96	74	58253	1275,96
	0,25	1361,71	3	48,8076	1349,51
	0,5	1475,87	0	0	1475,87
	0,75	1481,79	1	2,8304	1479,66
	1	1766,31	0	0	1766,31
	3	1509,18	0	0	1509,18
	5	1715,88	0	0	1715,88
	10	1763,81	0	0	1763,81
(400) C1.4.1	0	12027,9	305	227413	12027,9
	0,25	15145,4	27	991,255	14897,6
	0,5	16526,3	23	544,306	16254,2
	0,75	16557,6	22	522,069	16166
	1	15618,3	9	117,957	15500,4
	3	18807,5	5	18,827	18751,1
	5	17690,8	1	0,511386	17688,3
	10	18328,5	3	11,5043	18213,4
(1000) C1.4.1	0	65513,9	808	751213	65513,9
	0,25	90132,6	201	16925,8	85901,2
	0,5	110302	150	8512,76	106046
	0,75	114916	150	6491,03	110047
	1	101571	100	3756,61	97814,6
	3	121151	48	780,884	118808
	5	128280	35	651,157	125024
	10	129886	17	398,82	125898

8.2. Calibración del Esquema de Enfriamiento y Mecanismo de Estancamiento

Tomando como base la instancia mediana de 400 clientes con un valor fijo de $\alpha = 1,0$, se procedió a aislar el impacto de la temperatura inicial (T_0), la tasa de enfriamiento (COOLING_RATE) y el umbral de estancamiento (MAX_STAGNATION). Los resultados de estas tres fases consecutivas se consolidan en la Cuadro 2.

Cuadro 2: Resultados detallados del proceso secuencial de sintonización para la instancia de 400 clientes ($\alpha = 1,0$).

Parámetro Evaluado	Valor Objetivo	Nodos Penalizados	Penalización Tiempo	Distancia Recorrida
<i>Temperatura Inicial (T_0)</i>				
100	17088,6	11	183,465	16905,2
500	18108,8	11	253,658	17855,1
1000	18112,3	8	176,911	17935,4
5000	16022,8	11	82,1674	15940,6
10000	17445,3	13	196,747	17248,5
<i>Ratio de Enfriamiento</i>				
0.95	17321,4	15	103,058	17218,3
0.97	17360,2	13	184,298	17175,9
0.98	14271	15	219,675	14051,3
0.99	16022,8	11	82,1674	15940,6
0.995	17976,5	19	316,084	17660,4
<i>Umbral de Estancamiento</i>				
1000	26245,7	25	744,137	25501,5
2500	18621,3	20	411,262	18210
5000	14271	15	219,675	14051,3
10000	13410,7	7	41,2605	13369,4
15000	11965,3	3	33,4986	11931,8
20000	11175,8	1	12,1153	11163,6
30000	11175,8	1	12,1153	11163,6

Al analizar la **temperatura inicial**, se identificó que $T_0 = 5000$ proporciona el mejor desempeño balanceado, registrando la distancia recorrida más baja (15940,6). Valores inferiores (100 a 1000) provocan un enfriamiento temprano del sistema, transformando la búsqueda en un comportamiento codicioso que tiende a intensificar en las primeras regiones exploradas, quedando atrapado rápidamente en óptimos locales. Por el contrario, una temperatura excesivamente alta (10000), asemeja el proceso a una búsqueda totalmente aleatoria que no busca converger.

En cuanto al **ratio de enfriamiento**, la tasa de descenso de 0,98 destacó significativamente sobre las demás configuraciones. Un enfriamiento muy acelerado (0,95 o 0,97) reduce drásticamente la aceptación de soluciones no prometedoras en las etapas iniciales, impidiendo que el algoritmo diversifique la búsqueda en el espacio de soluciones. En el otro extremo, ritmos extremadamente lentos (0,995) permiten una tolerancia excesiva hacia soluciones perjudiciales, retrasando la estabilización del algoritmo y deteriorando la calidad del resultado final dentro del límite de iteraciones fijado. El factor de 0,98 ofrece el balance dinámico exacto para explorar y luego intensificar con éxito la solución.

Finalmente, la calibración del **umbral de estancamiento** (MAX_STAGNATION) reveló ser el componente con mayor impacto en el refinamiento de la solución. Al incrementar el umbral hacia un valor de 20000, se alcanzó el valor objetivo más bajo registrado (11175,8) junto con una drástica disminución en métricas como las penalizaciones temporales a un mínimo histórico

de 12,1153. Un umbral demasiado bajo (1000 a 5000) activa el recalentamiento con excesiva frecuencia, interrumpiendo constantemente el proceso de enfriamiento fino y desestabilizando la convergencia local. Permitir 20000 iteraciones asegura que el algoritmo agote rigurosamente la explotación fina de un vecindario válido a bajas temperaturas; una vez estabilizado, el recalentamiento al 30 % de la temperatura inicial proporciona el impulso exacto para saltar barreras hacia valles de búsqueda aún más prometedores y liberarse del estancamiento. Cabe notar que un aumento extra a 30000 iteraciones no produjo variaciones en el resultado, indicando que el algoritmo alcanzó su límite de convergencia para la semilla dada, manteniéndose estable.

9. Conclusiones

10. Uso LLMs

10.1. Claude Sonnet 4.6

1. **Herramienta utilizada:** Claude Sonnet 4.6 (Anthropic), versión gratuita accesible en claude.ai.
2. **Propósito de uso:** Búsqueda bibliográfica para el estado del arte sobre MDVRP-TW, revisión del modelo matemático y generación de ideas iniciales.
3. **Prompts relevantes:** Solicitud de bibliografía, resúmenes y explicaciones sobre MDVRP-TW, y consultas sobre restricciones, variables y formulaciones típicas en modelos matemáticos de VRP.
4. **Nivel de edición:** Alto. La bibliografía y sugerencias matemáticas fueron seleccionadas, revisadas y ajustadas manualmente basándose en su relevancia técnica.
5. **Validación:** Se realizó una validación mediante la lectura directa de los artículos fuente, consultas en foros especializados (StackExchange) y análisis propio.

10.2. ChatGPT 5.5

1. **Herramienta utilizada:** ChatGPT 5.5 (OpenAI), versión gratuita accesible desde chatgpt.com.
2. **Propósito de uso:** Apoyo en la implementación de software en C++, incluyendo corrección de tipado, refactorización, estructuración del proyecto, creación del Makefile y debugging de errores. Además, revisión de redacción, ortografía y cumplimiento de formato.
3. **Prompts relevantes:** “¿Cómo puedo corregir este error de tipado?”, “refactoriza este bloque de código para mejorar la legibilidad”, “recomiendame como estructurar mi proyecto...”, “genera un Makefile para este proyecto” y “revisa este párrafo para mejorar su redacción académica”.
4. **Nivel de edición:** Alto. El código no fue copiado íntegramente; se utilizó para corregir problemas puntuales, mejorar la arquitectura y resolver bugs. Las sugerencias fueron revisadas y adaptadas manualmente.
5. **Validación:** El código fue validado por juicio propio, contrastando el funcionamiento con la lógica requerida y observando el código. La redacción fue verificada manualmente contra la rúbrica.

10.3. Gemini

1. **Herramienta utilizada:** Gemini (Google), versión gratuita.
2. **Propósito de uso:** Contrastar información con la bibliografía, mejorar la redacción académica, formatear tablas de Excel a LaTeX, explicar restricciones matemáticas y revisar el cumplimiento de la rúbrica.
3. **Prompts relevantes:** “Mejora la redacción académica de este párrafo”, “convierte estos datos a tabla LaTeX”, “explica esta restricción matemática”, “¿es correcta esta afirmación según el paper?” y “revisa si este texto cumple la rúbrica”.
4. **Nivel de edición:** Alto. Todo contenido fue revisado, editado y verificado manualmente para asegurar precisión técnica antes de su inclusión.
5. **Validación:** Se contrastó la información contra las fuentes originales, aplicando juicio propio para la verificación final y asegurando la alineación con los requisitos del informe.

Referencias

- [1] T. Barros-Everett, E. Montero y N. Rojas-Morales. «Parameter Prediction for Metaheuristic Algorithms Solving Routing Problems using Machine Learning». En: *Applied Sciences* 15.6 (2025). DOI: [10.3390/app15062946](https://doi.org/10.3390/app15062946).
- [2] J.-F. Cordeau, G. Laporte y A. Mercier. «A unified tabu search heuristic for vehicle routing problems with time windows». En: *Journal of the Operational Research Society* 52.8 (2001), págs. 928-936. URL: <http://www.jstor.org/stable/822953> (visitado 15-05-2026).
- [3] J.-F. Cordeau y M. Maischberger. «A parallel iterated tabu search heuristic for vehicle routing problems». En: *Computers & Operations Research* 39.9 (2012), págs. 2033-2050. DOI: <https://doi.org/10.1016/j.cor.2011.09.021>.
- [4] G. B. Dantzig y J. H. Ramser. «The Truck Dispatching Problem». En: *Management Science* 6.1 (1959), págs. 80-91. URL: <https://www.jstor.org/stable/2627477> (visitado 14-05-2026).
- [5] Z. Lei y J. Hao. «A GPU-Accelerated Hybrid Method for a Class of Multi-Depot Vehicle Routing Problems». En: *arXiv preprint* (2026). URL: <https://arxiv.org/abs/2605.05208> (visitado 14-05-2026).
- [6] X. Li et al. «A Two-Stage Optimization Approach for Multi-Depot Vehicle Routing Problem with Time Windows». En: *2024 4th International Conference on Computer Communication and Artificial Intelligence (CCAI)*. 2024, págs. 540-545. DOI: [10.1109/CCAI61966.2024.10603115](https://doi.org/10.1109/CCAI61966.2024.10603115).
- [7] W. Liu, Y. Luo e Y. Yu. «An Adaptive Hybrid Genetic and Large Neighborhood Search Approach for Multi-Attribute Vehicle Routing Problems». En: *arXiv preprint* (2024). URL: <https://arxiv.org/abs/2402.18903> (visitado 14-05-2026).
- [8] Z. Luo, H. Qin y A. Lim. «Branch-and-price-and-cut for the multiple traveling repairman problem with distance constraints». En: *Journal of Combinatorial Optimization* 234.1 (2014), págs. 49-60. DOI: <https://doi.org/10.1016/j.ejor.2013.09.014>.
- [9] M. Polacek et al. «A Variable Neighborhood Search for the Multi Depot Vehicle Routing Problem with Time Windows». En: *Journal of Heuristics* 10 (2004), págs. 613-627. DOI: <https://doi.org/10.1007/s10732-005-5432-5>.
- [10] M. E. H. Sadati, D. Aksen y B. Çatay. «An efficient variable neighborhood search with tabu shaking for a class of multi-depot vehicle routing problems». En: *Computers & Operations Research* 133 (2021), pág. 105269. DOI: <https://doi.org/10.1016/j.cor.2021.105269>.

- [11] L. Shen, F. Tao y S. Wang. «Multi-Depot Open Vehicle Routing Problem with Time Windows Based on Carbon Trading». En: *International Journal of Environmental Research and Public Health* 15.9 (2018). DOI: [10.3390/ijerph15092025](https://doi.org/10.3390/ijerph15092025).
- [12] M. M. Solomon. «Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints». En: *Operations Research* 35.2 (1987), págs. 254-265. URL: https://www-labs.iro.umontreal.ca/~dift6751/paper_solomon.pdf (visitado 14-05-2026).
- [13] K. C. Tan et al. «Heuristic methods for vehicle routing problem with time windows». En: *Artificial Intelligence in Engineering* 15.3 (2001), págs. 281-295. DOI: [https://doi.org/10.1016/S0954-1810\(01\)00005-X](https://doi.org/10.1016/S0954-1810(01)00005-X).
- [14] T. Vidal et al. «A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows». En: *Computers & Operations Research* 40.1 (2013), págs. 475-489. DOI: <https://doi.org/10.1016/j.cor.2012.07.018>.
- [15] T. Vidal et al. «Heuristics for multi-attribute vehicle routing problems: A survey and synthesis». En: *European Journal of Operational Research* 231.1 (2013), págs. 1-21. DOI: <https://doi.org/10.1016/j.ejor.2013.02.053>.
- [16] S. Voigt et al. «Hybrid adaptive large neighborhood search for vehicle routing problems with depot location decisions». En: *Computers & Operations Research* 146 (2022), págs. 105856. DOI: <https://doi.org/10.1016/j.cor.2022.105856>.
- [17] S. Wang, W. Sun y M. Huang. «An adaptive large neighborhood search for the multi-depot dynamic vehicle routing problem with time windows». En: *Computers & Industrial Engineering* 191 (2024), págs. 110122. DOI: <https://doi.org/10.1016/j.cie.2024.110122>.